

Software Development and Integration

ASSIGNMENT 3

E-Scooter Data Audit

Maxim Mussin | CSE-2507M

Astana IT University

9 October 2026

Purpose and project context

This assignment develops a small scientific software component for the dissertation “Methods of detection and tracking of electric scooters in the traffic flow of the city”. Assignment 1 established work packages and computing resources; Assignment 2 selected an iterative process and a modular architecture [1]. The present work implements one component supporting those decisions: a command-line audit of detection annotations and dataset partitions.

Incorrect bounding boxes, broken annotation references and shared source recordings can invalidate a detector comparison. The implemented Python tool examines a documented subset of COCO metadata, checks partition overlap and exports structured findings. It operates before training and does not require a GPU, photographs or model weights for its public demonstration.

Completed deliverables

The submission contains a public Git repository, an MIT-licensed module, synthetic examples, 36 automated tests, a documented technology selection and a working GitHub Actions pipeline. The pipeline tests three environments, builds distributions, checks an installed wheel and delivers a downloadable artifact. A local audit of existing research splits supplies an additional integration example, with its limitations explicitly retained.

Repository: <https://github.com/izanamax/escooter-data-audit>

CI/CD configuration: versioned workflow file

Execution evidence: successful workflow run 37936258798

Report structure: development process (p. 2), technologies (p. 3), version control (p. 4), CI/CD (p. 5), scientific requirements (p. 6), implementation (p. 7), testing (p. 8), measured results (p. 9), conclusions and sources (p. 10).

1. Organising the development process

Development begins with the research failure that the software must prevent. Here the requirement is to detect identifiable annotation and partition defects before allocating compute to model training. It is narrower than developing the entire dissertation prototype. This boundary makes completion verifiable and keeps work consistent with the student-led process selected in Assignment 2 [1].

Stage	Work and completion evidence
Requirements	Define accepted COCO fields, error severity, split identities and output contract.
Design	Separate in-memory validation from file access, JSON decoding and CLI behavior.
Implementation	Build the audit library, command-line interface and synthetic examples.
Verification	Check correct and malformed inputs, exit codes, packaging and remote execution.
Delivery	Publish code, documentation, license, issues and a tested distribution artifact.
Maintenance	Record limitations and future extensions; rerun checks after changes.

The stages are revisited in short iterations. A new validation rule starts with an observable failure case, followed by implementation, local verification, a focused commit and remote checks. The acceptance criterion is a correct finding with a stable code, not merely successful program execution. For example, two different frame names from one declared recording must still trigger a sequence-overlap error.

The student owns implementation, tests and documentation. The supervisor's role remains scientific guidance and interpretation; this submission does not claim an independent code review. GitHub Issues record unresolved work with acceptance conditions. Source-recording provenance is an immediate research follow-up, while optional image decoding is a separate future extension.

A change is complete when its behavior is documented, relevant tests pass, inputs remain unchanged, and the package can be executed from its installation. This completion rule connects the task board to evidence. It also limits scope growth: new detector training, a web application and city-wide deployment are not necessary to deliver this data-audit module.

2. Selecting technologies and tools

Technology selection follows task fit, compatibility with existing research, installation effort, maintainability and the ability to verify results on a fresh machine. Speed of initial implementation alone is insufficient: a tool tied to one workstation would not provide a useful reproducibility check. The selected stack is deliberately small and CPU-based.

Decision	Reason and trade-off
Python 3.11+	Fits the existing detection project and handles nested JSON. Large datasets are still limited by in-memory loading.
Standard library	json, argparse, hashlib and pathlib cover the task without runtime dependencies.
unittest	Supports automated cases and isolated fixtures [6]. No additional testing package is needed.
Setuptools + build	Produce a standard wheel and source archive [8]. Build tooling adds dependencies only to packaging.
Git + GitHub Actions	Combine inspectable history, public hosting and verification beside the source code.

A pandas implementation would be useful for extensive tabular analysis, but the current checks traverse linked JSON records and do not need data frames. PyTorch and Ultralytics remain appropriate in the parent detector project; importing them here would increase installation and CI cost without helping metadata validation. A web framework would introduce interface and deployment work outside the module's requirements.

Pytest offers a useful alternative when richer fixtures and plugins are needed. The current suite is small enough for unittest and subprocess checks. The build environment pins build 1.3.0, setuptools 80.9.0, wheel 0.45.1 and their listed support packages. Runtime installation requires no external libraries. Python 3.11 and 3.12 are exercised in CI; other versions allowed by package metadata are not claimed as tested.

GitHub Actions was selected because the required repository is already on GitHub. GitLab CI could provide a comparable pipeline within GitLab, while Jenkins would require a separately managed service. For one researcher, maintaining that service would compete with scientific work. No extra GPU rental is needed for this assignment's verification.

3. Version control and repository practice

Version control records changes so that earlier states can be inspected and recovered. Git is distributed: a normal full clone contains repository history, and commits can be created locally. Subversion uses a central repository, so recording shared revisions depends on server access [2]. GitHub is a hosting and collaboration platform for Git repositories, not a competing version control system.

Dimension	Git	Subversion
History model	Distributed repository history	Central repository history
Offline work	Local commits and history inspection	Local edits; committing needs server access
Branch workflow	Local topic branches suit small experiments	Branches are repository paths
Fit here	Matches required GitHub delivery	Possible VCS, but not the required Git workflow

Observed repository history

The implementation is stored in izanamax/escooter-data-audit. The first commit, a914b47, adds the validation module, packaging configuration and examples. Commit 2b3075e adds 36 tests. Commit 151be74 adds the documented workflow and automated delivery. These are distinct implementation increments from this assignment, not an invented record of earlier dissertation development.

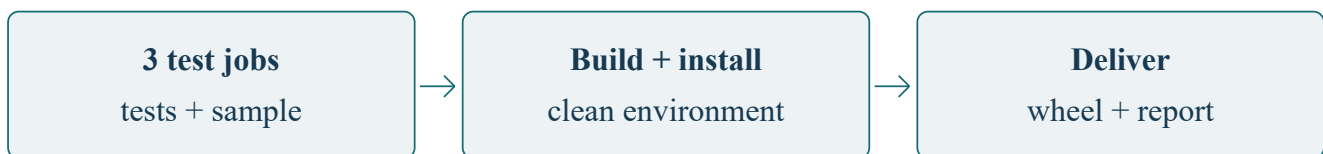
The repository contains a README with runnable commands, an MIT license for the new module, a requirements file for build tools, design notes, tests and public synthetic examples. The ignore rules exclude local environments, build outputs, model weights and local research results. The separate detector project's licensing and external dataset permissions are not changed by licensing this module. Issue #1 records the missing source-sequence provenance identified during integration. Issue #2 describes optional image-file verification. Both state future acceptance conditions and remain open. They expose outstanding work rather than presenting those capabilities as complete.

For subsequent changes, a short-lived branch and pull request can connect an issue, a readable diff and CI results. The initial delivery uses focused commits on main; branch protection, an independently reviewed pull request and enforced merge gates are not claimed. CI reports failures, while repository policy determines whether merging is blocked.

4. Continuous integration and delivery

Continuous integration combines frequent code integration with automated checks to expose defects early [4]. Continuous delivery prepares a verified package that a user can install; continuous deployment additionally automates deployment into an operating environment [5]. This project delivers a downloadable Python package. It has no production service to deploy.

GitHub Actions expresses the process as a versioned YAML workflow. Events start workflows; jobs run on runners; steps execute commands or reusable actions. Dependencies order jobs, and a matrix repeats verification across selected environments [3]. The submitted workflow responds to pushes, pull requests and manual dispatch.



The test stage runs 36 cases and a strict audit of the three synthetic splits on Ubuntu with Python 3.11, Ubuntu with Python 3.12, and Windows with Python 3.11. There are no runtime packages to install for those checks. Failure in any matrix entry prevents the dependent delivery job from proceeding.

The delivery job installs pinned build tools, creates a wheel and source archive, and installs the wheel into a new virtual environment. It changes out of the checkout directory before invoking the installed console command. This catches missing package files or a broken entry point that could be hidden by importing directly from the working tree.

Finally, the workflow uploads both distributions and the example audit report. The artifact name includes the full commit identifier and is retained for 30 days. GitHub artifacts preserve build outputs for later retrieval [9]. Downloading may require GitHub sign-in; after expiry, the public source and build instructions still support regeneration.

The workflow requests read access to repository contents, uses ten-minute job timeouts and pins reusable actions to full commit hashes. These choices reduce accidental drift and unnecessary permissions. Pinned actions still require deliberate updates. Hosted runner images can change, so the current pipeline demonstrates repeatable verification rather than guaranteed byte-identical builds.

5. Requirements for scientific software

Scientific software must support an interpretable result, not only produce output. In this dissertation, changing a dataset partition can affect the apparent quality of a detector. Consequently, the audit makes its input versions, checks and limitations visible. Its purpose is to prevent identifiable procedural errors before the larger model comparison.

Validity and data provenance

The tool checks required COCO detection fields and the relationships between images, annotations and categories. The COCO reference implementation provides the context for these linked records [7]. This implementation covers bounding boxes, not the full COCO specification: segmentation, keypoints, area and crowd flags are outside its validation contract. The README states this boundary explicitly.

Source-sequence membership matters because neighboring frames can share backgrounds and objects. A filename-only check cannot establish independence when different files come from one recording. The optional `source_sequence` field supplies that grouping. Missing values generate a warning by default; strict mode turns missing provenance into an error. A passing default audit with warnings must therefore not be described as complete leakage verification.

Environment and reproducibility

The CLI records the module version and SHA-256 digest of every supplied annotation file. The report has stable ordering and no timestamp. Repeating the command with identical named inputs and options produces the same content. The digest identifies input bytes; it does not certify correct labels. Git records the code revision, and the build requirements document the packaging environment.

For detector experiments, this audit report should be retained alongside the split manifest, model configuration, seed, dependencies and measured outputs. Those wider experiment records remain responsibilities of the parent research workflow. The audit neither reruns the models nor proves deterministic GPU training. A successful structural check and a credible scientific comparison are related but separate evidence.

Sharing and maintenance

Public examples contain synthetic metadata only. Raw traffic images and trained weights are unnecessary for running the package tests. Local audit reports may contain filenames or source identifiers supplied by a researcher and should be reviewed before sharing. Future rules should include a documented scientific reason and a regression case, so maintenance does not silently redefine what a passing report means.

6. Implemented module and integration

The package separates a pure validation function from its command-line wrapper. `audit_splits` accepts a mapping of split names to decoded datasets and returns a JSON-serializable report. The wrapper reads files, rejects malformed JSON, computes hashes, writes output and selects an exit code. Neither component edits the annotations.

Check group	Implemented behavior
Structure	Require three arrays; validate record objects and unique non-negative integer IDs.
Images and classes	Check declared dimensions, category names and safe relative filenames.
Annotations	Resolve image/category references; check finite xywh boxes and image bounds.
Dataset partitions	Compare normalized filenames, declared sequences and category mappings.
Reporting	Export counts, findings, input hashes, schema version and tool version.

COCO identifiers are local to each split. Reusing image ID 1 in two different files is valid and does not itself establish leakage. The module compares normalized filenames within a common dataset namespace, and uses declared source sequences when available. Slash direction is normalized, but filename comparison remains case-sensitive. Images without annotations are allowed because negative examples can be legitimate.

A bounding box is interpreted as `[x, y, width, height]`. Its coordinates must be finite numbers; origin coordinates must be non-negative; width and height must be positive. The box may touch an image boundary but may not exceed the declared dimensions. These checks identify malformed geometry without attempting to judge whether the box actually encloses an e-scooter.

Exit code 0 means the audit found no errors, possibly with warnings. Code 1 means validation completed with errors. Code 2 indicates a command, input or output problem. The CLI also prevents the report path from overwriting an input file. These distinctions let a training launcher stop on invalid data while preserving a diagnostic report.

Integration consists of running the command against the existing train, validation and test JSON exports before launching training. The module needs no changes to detector or tracker code. Its current limitations include no image decoding, no pixel-duplicate detection and no verification of the truthfulness of declared sequence identities.

7. Automated testing and verification

The test suite contains 36 unittest cases, with parameterized subcases for related invalid inputs. Unit tests exercise the in-memory API, while subprocess tests run the command-line interface. The unittest framework provides fixtures, assertions and isolated test execution [6]. Temporary directories keep file-based cases independent of research data.

Scenario	Expected observation
Valid data / boundary box	Pass with correct counts; a box touching the edge is accepted.
Malformed records or boxes	Structured error instead of a crash, including non-finite values.
Shared source recording	Fail even when individual frame filenames differ.
Repeated local IDs across splits	Pass when filenames and recording groups are distinct.
Missing source sequences	Warn normally; fail when strict mode is requested.
Unreadable or malformed JSON	Exit 2 with a readable error; no Python traceback.
Output equals input path	Reject the command and preserve original input bytes.

Other tests check category-map changes, duplicate IDs, invalid references, negative images, duplicate normalized filenames, deterministic reporting and input immutability. A very large integer in a box is treated as invalid rather than escaping through a numeric conversion error. These cases test observable behavior relevant to damaged annotation exports, not just whether functions can be called.

The public positive fixture contains seven images and seven boxes across three distinct synthetic recordings. Its strict audit must produce zero errors and warnings. The negative fixture has a box with negative width and an annotation pointing to an undeclared image. It must produce exit code 1 with exactly those two error categories.

Local Windows/Python 3.11 execution passed all 36 cases. Remote CI repeats the suite in three environments and verifies the installed package separately. This supports the documented behaviors on the tested inputs. No coverage percentage, proof of correctness for arbitrary datasets, or detector accuracy improvement is inferred from the test count.

8. Results and delivery evidence

The verified remote run is 37936258798, for commit 151be74299834326b04f46f0ec5db21f18c4d274. All three test jobs and the dependent build/install/delivery job completed successfully. The workflow produced one artifact containing the wheel, source archive and example audit report. This is observed execution evidence, rather than a proposed pipeline.

Verification	Observed result
Local test suite	36 tests passed on Windows / Python 3.11.
Remote matrix	36 tests passed in each of three environments.
Installed distribution	Clean-environment console command passed strict sample audit.
Synthetic inputs	7 images / 7 boxes: 0 errors, 0 warnings.
Invalid fixture	2 expected errors; exit code 1.

Integration with the existing research dataset

The same tool was run locally against the existing pilot split files without changing them. The supplied files contain 76 training images with 189 annotations, 15 validation images with 18 annotations, and 20 test images with 30 annotations: 111 images and 237 annotations in total. One training image has no annotations; this is allowed by the validation contract.

The audit found zero structural errors and three warnings, one for each split. Every image lacks `source_sequence` metadata. No normalized filename overlap or category-map mismatch was reported, but recording-level independence could not be verified. The absence of metadata does not prove that leakage occurred; it leaves that particular check unresolved. This finding is tracked in Issue #1.

The counts describe only the three inspected split exports, not every image in the broader research folders. The source README mentions an original 112-image pilot, so the report does not silently substitute that number for the observed 111. Resolving the provenance and membership difference belongs to a data audit before further comparisons.

The public repository provides the exact workflow configuration, tests, installation commands and documented limitations. Detailed local research findings remain in `local-results/pilot-audit.json`; only aggregate observations are presented here. The module reports no AP, IDF1 or other model-performance results.

9. Conclusions and references

The assignment connects development theory to an implemented scientific tool. A small validation contract, versioned code, behavioral tests and an installed-package check produced a usable component with inspectable delivery evidence. The main integration challenge was incomplete source-recording metadata: geometry checks could run, while sequence-level independence remained unverified. Warnings preserve this distinction without inventing provenance.

The next priority is to recover recording groups and confirm split membership before expensive detector comparisons. Optional image-file checks can follow as a separate extension. The delivered module, public repository and successful CI/CD run satisfy the present software task; completing the broader dissertation and resolving research-data provenance remain separate work.

References

- [1] Mussin, M. (2026). Assignment 1: Project Management and Financial Justification; Project Charter; Assignment 2: Methodology and Architecture Evaluation. Local dissertation planning documents.
- [2] Chacon, S., and Straub, B. Pro Git: About Version Control. Git project.
- [3] GitHub. Understanding GitHub Actions.
- [4] GitHub. Continuous Integration.
- [5] GitHub. Continuous Deployment.
- [6] Python Software Foundation. unittest: Unit Testing Framework.
- [7] COCO Consortium. COCO Python API: coco.py.
- [8] Python Packaging Authority. Packaging Python Projects.
- [9] GitHub. Store and Share Data with Workflow Artifacts.

Online documentation consulted for this submission on 9 October 2026. Implementation details, test counts and audit results are based on the submitted source and observed runs; the external references provide conceptual and API context.